An Omega-Diagonal Forest Reflection Above BusyLean Contender 10

A computable Lean proposal with an explicit comparison architecture

Research proposal prepared for the BusyLean project

October 12, 2026

Abstract

BusyLean asks for closed computable natural numbers in Lean, together with kernel-checkable proofs that each new entry exceeds its predecessor. The current repository entry, Contender 10, reflects kind-polymorphic System F_ω programs over Contender 9's family. We propose Contender 11, which combines that language-level reflection with the repository's strongest existing structural amplifier: forest reflection along the self-generated W-tree index that joins every tree denoted by the W-language. One traversal is promoted to a function operator $\mathcal{R}$. We then define a self-accelerated tower $\mathcal{T}_0 = B$ and $\mathcal{T}_{k+1}(n) = \mathcal{R}(\mathcal{T}_k)(\mathcal{T}_k(n))$, where $B(n) = F_{10}(F_{10}(n))$ is Contender 10's construction as a family. The new diagonal is

$$F_{11}(n) = \mathcal{T}_{B(n)+1}(B(n)), \qquad C_{11} = F_{11}(F_{11}(42)).$$

Every component is computable in principle. The comparison avoids any invalid monotonicity assumption about oracle reflection: it exhibits the seed explicitly inside the index tree, proves that every tower successor is larger by the round's input bound, and derives $C_{10} < F_{11}(42) < C_{11}$. The package contains Lean definitions, proposed proofs, a transitive dependency audit, a clean-build checker, and this article. Proof-theoretic size claims are deliberately separated from the exact formal inequality.

Verification status. The article has been compiled and visually checked. The Lean source has passed a conservative source-level audit, but the execution environment used to prepare this package did not contain Lean 4.34.0. Consequently, the modules in this archive are a serious, integration-ready *proposal*, not a claim of completed kernel verification. The bundled `check11.py` and `Contender11Audit.lean` specify the exact final checks.

Contents

1 Problem, constraints, and baseline **4**
- 1.1 The BusyLean game . . . 4
- 1.2 The current incumbent . . . 4
- 1.3 Available machinery that has not yet been combined . . . 4

2 Design criteria **5**

3 Reused formal ingredients **5**
- 3.1 Countable oracle families . . . 5
- 3.2 The self-generated W-tree index . . . 6
- 3.3 Kind-polymorphic program evaluation . . . 6

4 The construction **6**
4.1 One family-reflection step . . . 6
4.2 Forest reflection . . . 7
4.3 A complete round . . . 7
4.4 The predecessor family . . . 7
4.5 Self-accelerated iteration . . . 7
4.6 The diagonal and final number . . . 8

5 Exact comparison architecture **8**
5.1 Local maximum inequalities . . . 8
5.2 Children are below parents . . . 8
5.3 A full round explicitly contains its seed . . . 9
5.4 Every tower successor is larger . . . 9
5.5 The predecessor family exceeds its input . . . 10
5.6 The diagonal dominates the predecessor family . . . 10
5.7 The required comparison . . . 10

6 Structural margins beyond the required inequality **10**
6.1 Every round beats the seed pointwise . . . 11
6.2 Every W-definable tree is eventually absorbed . . . 11
6.3 Every visible accepted kind-polymorphic program is absorbed . . . 11

7 Computability and totality **11**
7.1 Finite syntax enumeration . . . 11
7.2 Total program evaluation . . . 11
7.3 Finite maxima . . . 11
7.4 Structural recursion over W-trees . . . 12
7.5 Finite tower recursion . . . 12
7.6 Closedness . . . 12

8 Why the growth jump is significant **12**
8.1 Three nested amplifiers . . . 12
8.2 A cautious operator heuristic . . . 12
8.3 Formal lower bounds that do not rely on the heuristic . . . 13

9 Why monotonicity is deliberately avoided **13**

10 Lean engineering and audit plan **14**
10.1 Module split . . . 14
10.2 Forbidden-feature source audit . . . 14
10.3 Kernel-level dependency closure . . . 14
10.4 Axiom reports . . . 14
10.5 Time budgets . . . 14
10.6 Likely elaboration risks . . . 14

11 Alternatives considered **15**
11.1 A single complete round . . . 15
11.2 Same-input iteration . . . 15
11.3 Pointwise maxima of child stages . . . 15
11.4 A universe ladder . . . 15
11.5 Dependent large elimination . . . 15

12 Integration procedure **16**

13 Further research **16**
13.1 Formalize an abstract reflection-round interface . . . 16
13.2 Prove quantitative embedding theorems . . . 16
13.3 Study monotone sublanguages . . . 17
13.4 Calibrate the round operator . . . 17
13.5 Build the stratified kind-universe successor . . . 17
13.6 Mechanize the metatheory of the audit itself . . . 17

14 Conclusion **18**

A Core Lean definition **18**

B Theorem dependency graph **19**

C Reproducibility commands **19**

D Acceptance checklist **19**

1 Problem, constraints, and baseline

1.1 The BusyLean game

The public BusyLean repository is a Lean 4 continuation of Cody Roux's "name the biggest number" contest. A successful entry is not merely a concise large-number notation. It must be a closed term of type `Nat`, computable in principle, and it must be accompanied by a formal strict inequality against the preceding entry. The repository rules further prohibit escape hatches such as custom axioms, `noncomputable`, `partial`, `unsafe`, opaque stand-ins, external implementations, and `native_decide`. The only allowed logical axioms are `propext` and `Quot.sound`; the current target is Lean 4.34.0. Definition and proof modules are also subject to separate time budgets. These constraints make the problem a useful synthesis of proof theory, programming-language semantics, formal verification, and careful software engineering [1, 6].

The no-laziness rule matters. A successor may use the preceding *construction* as a seed or oracle for a qualitatively stronger operation, but it may not simply mention the previous constant and add one, exponentiate it, or wrap it in a fixed arithmetic expression. A convincing successor must expose a new source of growth and prove that the predecessor occurs inside it.

1.2 The current incumbent

At the inspected repository revision, Contender 10 is

```
def family9 (n : Nat) : Nat := F9 (F9 n)
def F10 (n : Nat) : Nat := Fk.reflectFk family9 n
def contender_10 : Nat := F10 (F10 42)
```

Here `Fk.reflectFk` takes a finite maximum over accepted, closed programs of kind-polymorphic System F_ω at a syntax/fuel budget n, evaluated relative to an oracle f, and also exceeds $f(n)$ and n. The repository has a strong-normalization theorem for the object language, so program evaluation is a total Lean function rather than a bounded simulation or a semantic oracle. The accepted comparison proves `contender_9_lt_contender_10` [4, 5].

Contender 10 is already far beyond elementary or primitive-recursive large-number games. The problem is therefore not to add a familiar fast-growing function. A meaningful successor should exploit a structural operation that is not already compressed into the single bounded reflection step.

1.3 Available machinery that has not yet been combined

Two ingredients coexist in the repository:

1. **Kind-polymorphic reflection.** Contender 10 can normalize and decode accepted programs of a language stronger than plain System F_ω, with a computable oracle.

2. **Self-generated forest reflection.** Contenders 6 and 9 use a well-founded, countably branching tree `index`. Its children join all W-trees denoted by programs of the repository's W-language. At every node, the reflected program sees the family of child stages as an oracle, so it can select a child dynamically instead of losing information in a pointwise maximum [2, 3].

The repository's own roadmap observes that forest reflection can be applied to new languages. The first move of the present proposal is precisely to apply the Contender 9 amplifier to Contender 10's stronger reflected language. The second move is to iterate the resulting complete operation and diagonalize over the number of iterations.

2 Design criteria

A serious successor should satisfy more than the final inequality. We impose the following criteria.

C1: closed computability. The final declaration must reduce, in principle, to a natural number by Lean's ordinary evaluator. All recursion must be structural or justified by an already formalized well-founded argument.

C2: no predecessor constant. The definition may refer to `F10` and the preceding construction as a function, but its transitive declaration closure must not contain `contender_10` or any earlier contender constant.

C3: qualitative amplification. The new operation must do more than evaluate the old family at a larger fixed argument. It should expand the reflected program class, the iteration structure, or both.

C4: explicit comparison. Reflection relative to an oracle is generally not monotone: a program may branch on an oracle value, so pointwise larger oracle answers can produce a smaller result. Every inequality must therefore come from an explicit occurrence in a finite maximum or from a child embedded in a known tree.

C5: modest proof surface. The construction should reuse the repository's difficult semantic theorem, namely strong normalization for kind-polymorphic System F_ω, instead of introducing a new object language whose metatheory cannot be completed within the contest budgets.

C6: honest calibration. Lean will certify the strict numerical comparison. It will not automatically certify an informal ordinal assignment. The article must distinguish exact formal statements from proof-theoretic heuristics.

These criteria rule out a number of superficially attractive definitions. For example, $2^{C_{10}}$ is computable and larger, but it violates C2 and C3. A fixed tenfold iterate of `F10` uses the construction rather than the constant, but still offers no new language or reflection structure. At the other extreme, formalizing a Calculus of Inductive Constructions with large elimination could provide a dramatic language-level jump, but violates C5 for a compact successor proposal.

3 Reused formal ingredients

3.1 Countable oracle families

A forest node has countably many children. The reflected language accepts one unary oracle, so Contender 9 uses the computable coding

$$\langle i, m\rangle_2 = 2^i(2m+1).$$

The decoder takes the 2-adic valuation to recover i and then recovers m. The theorem `unpair2_code` proves that the code is inverted exactly. Thus for a family $fs : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$,

$$\texttt{packFamily}\ fs\ (2^i(2m+1)) = fs(i, m).$$

The coding is not itself the source of growth. Its role is information preservation: a program can compute a child index and an argument at run time, then call that exact child stage.

3.2 The self-generated W-tree index

The repository's W-language contains W-types over finite types and a total interpreter. A program that denotes a tree of type $\mathsf{W}\mathbb{N}$ is converted to a Lean tree by `defTree`; a non-tree value defaults to a leaf. For each syntax depth k, `treesOfDepth k` is a node whose children enumerate the trees denoted at that depth. The global index is

$$\texttt{index} = \texttt{node}\ (k \mapsto \texttt{treesOfDepth}\ k).$$

It is therefore a well-founded tree in Lean, yet it joins every W-tree that the object language denotes at some finite syntax depth. The key formal embedding used repeatedly in BusyLean is

$$\text{leaf} \prec \texttt{treesOfDepth 0} \prec \texttt{index},$$

where each $\prec$ is an actual parent-child relation visible at every sufficient input. The first enumerated depth-zero tree is definitionally a leaf, recorded by `first_tree_leaf`.

3.3 Kind-polymorphic program evaluation

The namespace `BusyLean.Fk` provides:

- finite enumerations `kindsUpTo`, `tysUpTo`, and `ctmsUpTo`;
- a sound fuel-bounded checker;
- a strong-normalization theorem for accepted erased terms;
- a total normalizer and Church-numeral decoder; and
- `fvalue f fuel t`, the numerical value of an accepted program relative to the unary oracle f, with zero as the rejection/default value.

This is the expensive semantic core. The proposal treats it as a verified component and builds a stronger reflection geometry around it. Reducibility candidates are the classical technique behind strong-normalization arguments for polymorphic calculi; related modular normalization results explain why language boundaries in the lambda cube are central to proof-theoretic comparisons [8].

4 The construction

4.1 One family-reflection step

Let $fs : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ be a countable family. Define

$$M_{fs}(n) = \max\left(\max_{i \le n} fs(i,n),\ \max_{t \in \mathrm{CTm}_{\le n}} \mathrm{value}_{\mathrm{F}_\kappa}(\mathrm{pack}(fs), n, t)\right),$$

$$\mathrm{reflect}_{11}(fs, n) = n + 1 + M_{fs}(n).$$

The Lean definition is:

```
def reflect11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum
      (Fk.fvalue (packFamily fs) n)
      (Fk.ctmsUpTo n))
```

The added $n + 1$ is not intended as a growth amplifier. It provides a robust strict inequality above the input and above either maximum, simplifying every subsequent comparison.

4.2 Forest reflection

The family step is iterated structurally over a W-tree:

```
def stage11 (seed : W.Family) : BigNumber.WTree Nat → Nat → Nat
  | .leaf   => reflect11 seed
  | .node g => reflect11 (fun i => stage11 seed (g i))
```

At a leaf, the reflected language receives the seed family. At a node, it receives the family of child stages. Notice what is *not* done: the children are not collapsed to a pointwise maximum before program evaluation. The program sees them through `packFamily`, so it can choose a child based on intermediate computation.

4.3 A complete round

For a unary seed $f : \mathbb{N} \to \mathbb{N}$, repeat it across the leaf family and traverse the full self-generated index:

```
def round11 (f : Nat → Nat) (n : Nat) : Nat :=
  stage11 (fun _ => f) index n
```

We write this function operator as $\mathcal{R}(f)$. A complete round is already a substantial candidate successor: it reflects the kind-polymorphic language at every node along every W-language-definable tree embedded in `index`. However, the operation itself is computable and strictly seed-increasing, so it can be iterated.

4.4 The predecessor family

The rule against direct use of the old constant is respected by exposing the entire preceding construction as a function:

```
def family10 (n : Nat) : Nat := F10 (F10 n)
```

Write $B(n) = F_{10}(F_{10}(n))$. Then $B(42)$ is definitionally equal to Contender 10, but the declaration `contender_10` itself does not occur in the new definition.

4.5 Self-accelerated iteration

A naive iteration would set $S_{k+1}(n) = \mathcal{R}(S_k)(n)$. We use a stronger and still elementary recursion:

$$\mathcal{T}_0(n) = B(n), \qquad \mathcal{T}_{k+1}(n) = \mathcal{R}(\mathcal{T}_k)(\mathcal{T}_k(n)).$$

Thus the preceding stage plays two roles. It is the oracle seed throughout the next forest, and its value becomes the next syntax/fuel/input budget. The Lean code is:

```
def tower11 : Nat → Nat → Nat
  | 0     => family10
  | k + 1 => fun n => round11 (tower11 k) (tower11 k n)
```

This recursion is structural on k. It does not attempt to evaluate the huge natural during type checking. As a mathematical function it is total for every finite k and n.

4.6 The diagonal and final number

The number of complete rounds should not be a small numeral chosen by the paper's author. We take it from the preceding construction itself:

```
def F11 (n : Nat) : Nat :=
  tower11 (family10 n + 1) (family10 n)

def contender_11 : Nat := F11 (F11 42)
```

Equivalently,

$$F_{11}(n) = \mathcal{T}_{B(n)+1}(B(n)), \qquad C_{11} = F_{11}(F_{11}(42)).$$

The family B is provably larger than its input, so the iteration depths $B(n)+1$ are unbounded. In this precise but modest sense, F_{11} is an ω-diagonal through all finite numbers of complete rounds. It is not a literal transfinite recursion of length ω; every evaluation uses a finite but extraordinarily large number of stages.

5 Exact comparison architecture

This section gives the mathematical content mirrored by `Comparison11.lean`. The difficult metatheory is already inside `Fk.fvalue`; the new comparison is elementary arithmetic plus explicit child inclusions.

5.1 Local maximum inequalities

Lemma 5.1 (Visible oracle values)**.** *If $i \le n$, then*

$$fs(i,n) < \mathrm{reflect}_{11}(fs,n).$$

Proof. The prefix maximum contains $fs(i,n)$. It is at most the outer maximum, which is strictly below $n+1$ plus that maximum. □

Lemma 5.2 (Accepted program values)**.** *If t belongs to `Fk.ctmsUpTo n`, then*

$$\texttt{Fk.fvalue}\ (\texttt{packFamily}\ fs)\ n\ t < \mathrm{reflect}_{11}(fs,n).$$

Proof. The finite list maximum contains the displayed value, and the same arithmetic argument applies. □

Lemma 5.3 (Input bound)**.** *For every fs and n,*

$$n < \mathrm{reflect}_{11}(fs,n).$$

The Lean proofs reuse the repository's lemmas for prefix and list maxima. No normalization or evaluation of a contender is required.

5.2 Children are below parents

At a node `node` g, the oracle family is $i \mapsto$ `stage11` *seed* $(g(i))$. Therefore, when $i \le n$, the visible oracle lemma yields

$$\mathrm{stage}_{11}(seed, g(i), n) < \mathrm{stage}_{11}(seed, \mathrm{node}(g), n).$$

This is `stage11_child_lt`. The input bound similarly lifts from one reflection step to every tree.

5.3 A full round explicitly contains its seed

The critical proof is not a monotonicity theorem.

Theorem 5.4 (Seed inclusion)**.** *For every $f : \mathbb{N} \to \mathbb{N}$ and every n,*

$$f(n) < \mathcal{R}(f)(n).$$

Proof. Use three strict inequalities.

First, at the leaf the constant family $i \mapsto f$ visibly contains $f(n)$:

$$f(n) < \mathrm{stage}_{11}(i \mapsto f, \mathrm{leaf}, n).$$

Second, the first child of `treesOfDepth 0` is definitionally that leaf, so child domination gives

$$\mathrm{stage}_{11}(i \mapsto f, \mathrm{leaf}, n) < \mathrm{stage}_{11}(i \mapsto f, \texttt{treesOfDepth 0}, n).$$

Third, `treesOfDepth 0` is the zeroth child of `index`, hence

$$\mathrm{stage}_{11}(i \mapsto f, \texttt{treesOfDepth 0}, n) < \mathrm{stage}_{11}(i \mapsto f, \texttt{index}, n) = \mathcal{R}(f)(n).$$

Transitivity completes the proof. □

The corresponding Lean theorem is:

```
theorem seed_lt_round11 (f : Nat → Nat) (n : Nat) :
    f n < round11 f n := by
  have h₁ : f n < stage11 (fun _ => f) .leaf n := ...
  have h₂ : stage11 (fun _ => f)
      (defTree (nthD (BigNumber.allTerms 0) 0 .zero)) n <
      stage11 (fun _ => f) (treesOfDepth 0) n := ...
  have h₃ : stage11 (fun _ => f) (treesOfDepth 0) n <
      stage11 (fun _ => f) index n := ...
  rw [first_tree_leaf] at h₂
  exact Nat.lt_trans h₁ (Nat.lt_trans h₂ h₃)
```

This explicit path is robust under nonmonotone oracle behavior. A reflected program may do anything with other oracle entries; the finite maximum still contains the particular child needed for the proof.

5.4 Every tower successor is larger

Theorem 5.5 (Strict tower)**.** *For all $k, n \in \mathbb{N}$,*

$$\mathcal{T}_k(n) < \mathcal{T}_{k+1}(n).$$

Proof. By definition,

$$\mathcal{T}_{k+1}(n) = \mathcal{R}(\mathcal{T}_k)(\mathcal{T}_k(n)).$$

Every round exceeds the numerical input at which it is evaluated, so

$$\mathcal{T}_k(n) < \mathcal{R}(\mathcal{T}_k)(\mathcal{T}_k(n)).$$

□

It follows by ordinary induction and transitivity that every positive tower level exceeds the base family at the same input:

$$B(n) < \mathcal{T}_{k+1}(n).$$

This result is formalized as `family10_lt_tower11_succ`.

5.5 The predecessor family exceeds its input

Contender 10's local input theorem says $n < F_{10}(n)$. Applying it twice gives

$$n < F_{10}(n) < F_{10}(F_{10}(n)) = B(n).$$

Thus `input_lt_family10` requires no monotonicity of F_{10}: it uses the input theorem once at n and once at the concrete input $F_{10}(n)$.

5.6 The diagonal dominates the predecessor family

Theorem 5.6 (Family domination)**.** *For every n,*

$$B(n) < F_{11}(n).$$

Proof. First apply the preceding input bound at input $B(n)$:

$$B(n) < B(B(n)).$$

The right-hand side is the base level $\mathcal{T}_0(B(n))$. Since $B(n) + 1$ is positive, strict tower iteration then yields

$$B(B(n)) < \mathcal{T}_{B(n)+1}(B(n)) = F_{11}(n).$$

□

Combining this theorem with $n < B(n)$ gives the useful diagonal bound $n < F_{11}(n)$.

5.7 The required comparison

At 42, $B(42)$ is definitionally Contender 10. Therefore

$$C_{10} = B(42) < F_{11}(42).$$

The diagonal input theorem, now at the concrete input $F_{11}(42)$, gives

$$F_{11}(42) < F_{11}(F_{11}(42)) = C_{11}.$$

Hence:

Theorem 5.7 (Proposed Lean theorem)**.**

$$C_{10} < C_{11}.$$

In the package this is the declaration `contender_10_lt_contender_11`*.*

The proof term never needs to normalize either number. It compares their construction trees symbolically.

6 Structural margins beyond the required inequality

A contest entry is easier to assess when it proves more than one isolated comparison. The package includes three structural margins.

6.1 Every round beats the seed pointwise

The theorem `seed_lt_round11` holds for an arbitrary unary function, not just `family10`. Thus $\mathcal{R}$ is inflationary on the diagonal values needed by the construction, even though no general pointwise monotonicity claim is made.

6.2 Every W-definable tree is eventually absorbed

Suppose a W-language term t occurs in the finite enumeration at depth k. The repository lemma `exists_nthD_of_mem` supplies an index i at which t occurs. For every $n \geq k + i$, both k and i are visible child indices. Two applications of child domination establish

$$\mathrm{stage}_{11}(i \mapsto f, \mathrm{defTree}(t), n) < \mathcal{R}(f)(n).$$

This is formalized as `defined_tree_eventually_lt11`. It records the reason `index` is stronger than a single hand-written tree: every tree denoted by the object language is eventually present.

6.3 Every visible accepted kind-polymorphic program is absorbed

At the root of `index`, the child family is

$$i \longmapsto \mathrm{stage}_{11}(j \mapsto f, \texttt{treesOfDepth}(i)).$$

For every accepted term in `Fk.ctmsUpTo n`, the program maximum inside the root reflection contains its value relative to this exact family. The theorem `fvalue_lt_round11` exposes that fact. It is the language-level analogue of defined-tree absorption.

Together these statements show that the proposal is not merely a numerically larger wrapper. A round absorbs both the relevant program enumeration and the relevant tree enumeration, while preserving dynamic access to the family of sub-stages.

7 Computability and totality

The phrase "computable in principle" needs a concrete decomposition.

7.1 Finite syntax enumeration

For each natural budget n, `Fk.ctmsUpTo n` is a finite list generated by primitive recursion. The list may be astronomically large, but finiteness, not feasibility, is the contest requirement.

7.2 Total program evaluation

For each listed term, the checker either rejects it, returning 0, or supplies a typing derivation at the Church-numeral type. The repository's reducibility argument proves strong normalization of the erasure relative to every total oracle function. The normalizer is then a well-founded recursive function and the decoder is total. No halting oracle or Busy Beaver occurs.

7.3 Finite maxima

The maximum of a finite list of natural values is computable. Prefix maxima over indices $0, \ldots, n$ are primitive recursive.

7.4 Structural recursion over W-trees

The function `stage11` is defined by the recursor of the inductive W-tree type. Although a node has a function of countably many children, every recursive call is on an immediate subtree, and Lean's structural recursion principle certifies totality.

7.5 Finite tower recursion

For each natural k, `tower11 k` is obtained by k structural successor steps. The fact that $k = B(n)+1$ is too large to realize physically does not make it noncomputable. The recursion is finite for every input.

7.6 Closedness

All parameters are bound before the final declaration. The numeral 42, the existing computable functions, the syntax enumerators, and the total normalizers determine one closed `Nat`. There are no external files, random choices, unspecified witnesses, or semantic oracles in its definition.

Remark 7.1. Practical evaluability is intentionally absent. Even very small budgets for the underlying syntax enumerations can be costly. BusyLean's rule is constructive definability plus formal comparison, not decimal expansion or execution at the contest input.

8 Why the growth jump is significant

8.1 Three nested amplifiers

The construction combines three different operations rather than repeatedly applying one arithmetic function.

1. **Language reflection.** At each node, take a maximum over accepted kind-polymorphic System F_ω programs with an oracle.
2. **Tree-indexed reflection.** Arrange these node reflections along a self-generated W-tree that contains every W-language-definable tree.
3. **Self-accelerated finite iteration.** Feed the entire preceding round back as the next oracle and use its current value as the next budget; then choose the iteration count from the preceding contender family.

Contender 10 uses the first operation once. A single `round11` combines the first two. The tower repeatedly applies the combined operation, and F_{11} diagonalizes through a predecessor-sized number of repetitions.

8.2 A cautious operator heuristic

Let Δ denote, only heuristically, the proof-theoretic increment associated with one complete F_κ forest-reflection round over `index`. If ordinary oracle reflection behaved like ordinal addition, then repeated rounds would suggest a progression

$$\beta, \quad \beta + \Delta, \quad \beta + \Delta \cdot 2, \quad \ldots,$$

and a diagonal through unbounded finite counts would point toward a limit near $\beta + \Delta \cdot \omega$. The self-feeding input $\mathcal{T}_k(n)$ makes the numerical growth substantially more aggressive than this schematic index progression alone.

This picture is useful intuition, but it is not a theorem for the implemented operator. The reflected program can inspect exact oracle values, and the self-generated tree index mixes syntax and semantic evaluation. A defensible ordinal calibration would require a separate interpretation theorem connecting the Lean hierarchy to a recognized reflection calculus or ordinal notation system.

Modern proof theory confirms in many settings that iterated reflection is not a cosmetic operation: reflection ranks can recover proof-theoretic ordinals, and iterations of uniform reflection produce systematic ordinal progressions [9, 10]. Those results motivate the design, but they do not identify the exact ordinal of this concrete BusyLean operator.

8.3 Formal lower bounds that do not rely on the heuristic

Even without an ordinal assignment, the package proves the following exact facts, subject to kernel verification:

- $B(n) < F_{11}(n)$ for every input n;
- $\mathcal{T}_k(n) < \mathcal{T}_{k+1}(n)$ for every finite level;
- every W-language-definable tree stage is eventually below a round;
- every visible accepted kind-polymorphic program value is below a round; and
- $C_{10} < C_{11}$.

These are sufficient to justify "significantly exceeds" in the contest sense: the new number arises from a strict hierarchy of complete reflection rounds, not from a one-off arithmetic padding term.

9 Why monotonicity is deliberately avoided

It is tempting to assert

$$f \le g \quad \Longrightarrow \quad \mathcal{R}(f) \le \mathcal{R}(g).$$

For an arbitrary reflected programming language this is false without strong restrictions. A program can query the oracle and return 100 when the answer is zero but return 0 otherwise. Increasing the oracle can therefore lower that program's output. Taking a maximum over many programs does not automatically repair the issue at every finite budget.

The comparison proofs use only four safe mechanisms:

1. membership in a concrete finite maximum;
2. membership in a concrete prefix maximum;
3. a concrete parent-child edge in a W-tree; and
4. transitivity of strict order on natural numbers.

This proof style is one of the proposal's strongest engineering properties. It localizes every inequality and makes the argument insensitive to complicated control flow inside the reflected language.

10 Lean engineering and audit plan

10.1 Module split

The proposed definition module imports only `BusyLean.Contender10`. It contains the six computable declarations `reflect11`, `stage11`, `round11`, `family10`, `tower11`, `F11`, and `contender_11`. The comparison module imports the definition and `BusyLean.Comparison10`; this keeps proof-only dependencies out of the number's definition module.

10.2 Forbidden-feature source audit

The bundled `static_audit.py` removes nested Lean comments and string contents, then rejects occurrences of `sorry`, `admit`, custom `axiom`, `noncomputable`, `partial`, `unsafe`, `opaque`, `implemented_by`, `extern`, `native_decide`, and `Classical.choice`. It also rejects any direct earlier contender name in the executable text of the definition module and enforces a 100-column source limit.

This check is intentionally conservative and intentionally insufficient. Text search cannot establish type correctness, termination, or transitive axiom usage.

10.3 Kernel-level dependency closure

`Contender11Audit.lean` copies the repository's environment traversal. For a root declaration it follows every constant in declaration types and bodies, then fails if any forbidden earlier contender constant appears. The intended assertion is:

```
#assert_not_in_closure11 BusyLean.contender_11 :
  BusyLean.contender_10 BusyLean.contender_9 BusyLean.contender_8
  BusyLean.contender_7 BusyLean.contender_6 BusyLean.contender_5
```

This is stronger than checking the source file for a name. It detects an indirect dependency introduced through helper declarations.

10.4 Axiom reports

The audit prints axioms for the new number, required comparison, tower theorem, family theorem, tree absorption theorem, and program absorption theorem. The checker parses Lean's output and accepts only `propext` and `Quot.sound`, matching the current repository policy.

10.5 Time budgets

The proposed `check11.py` builds `BusyLean.Comparison10` first as the accepted predecessor, then times `BusyLean.Contender11` against the 15-second definition budget and `BusyLean.Comparison11` against the 60-second proof budget. It finally runs the audit. Actual timings must be recorded on a reasonable machine before acceptance; none are fabricated here.

10.6 Likely elaboration risks

The mathematical proof uses only existing theorem interfaces, but this package was prepared without a local Lean executable. The likely failure modes are small elaboration issues rather than conceptual gaps:

- definitional reduction may need an explicit `simpa [F11]`;
- the root reduction in `fvalue_lt_round11` may need `unfold round11 index stage11`;
- the final comparison may prefer an explicit `change`; or
- a theorem name exported transitively by `Comparison10` may need an additional direct import.

The package's `PROOF_OBLIGATIONS.md` provides fallback proof forms for these points. None changes the construction or argument.

11 Alternatives considered

11.1 A single complete round

The minimal successor

$$C' = \mathcal{R}(B)(\mathcal{R}(B)(42))$$

would be easy to compare and already add forest reflection to Contender 10. It was rejected as insufficiently ambitious because $\mathcal{R}$ is itself a total, strictly seed-increasing operator. Once that lemma is available, a finite tower and diagonal cost little additional proof complexity.

11.2 Same-input iteration

One could define $S_{k+1}(n) = \mathcal{R}(S_k)(n)$. The selected recurrence evaluates at $S_k(n)$ instead. The stronger recurrence has an equally simple proof because every round exceeds its input. It also makes the syntax budget, type-checker fuel, and visible child prefix grow with the previous stage.

11.3 Pointwise maxima of child stages

Replacing the oracle family by $\max_{i \le n} \mathrm{child}_i(n)$ would simplify types but destroy dynamic child selection. A program would learn only one aggregate value, not the identity and behavior of a selected child function. The existing packed-family interface is strictly more informative.

11.4 A universe ladder

The BusyLean roadmap proposes stratified universes of kinds above the current kind-polymorphic language. That is the most natural next language-level research program. A complete entry would require syntax, substitution, kind-checking, conversion, candidates indexed by universe level, adequacy, a sound checker, and a normalizer. It may eventually dominate the present amplifier by a qualitative margin, but it is not a low-risk continuation of the current formalization.

11.5 Dependent large elimination

A CIC-like language with large elimination can let run-time naturals control type structure and may surpass fixed finite-order hierarchies more directly. Its realizability or normalization proof is substantially more involved. The present proposal should be viewed as an immediately available successor and a platform from which that larger project can be funded by a still larger oracle family.

12 Integration procedure

The archive is arranged so it can be copied into a BusyLean checkout without replacing existing files.

1. Copy `Contender11.lean` and `Comparison11.lean` into `BusyLean/`.
2. Copy `Contender11Audit.lean` into `test/`.
3. Copy `check11.py` into `scripts/`.
4. Append imports of the two new modules to `BusyLean.lean`.
5. Run `python3 scripts/check11.py` from the repository root.
6. If the build passes, inspect the axiom reports and closure message rather than relying only on the script's final line.
7. Add the supplied README section and update the main article only after timings and kernel verification are known.

A unified patch is included for convenience. Because repository development may continue, applying the patch with `git apply -check` before mutation is recommended.

13 Further research

13.1 Formalize an abstract reflection-round interface

The comparison pattern is independent of the details of F_κ. An abstract structure could package:

- a finite program enumeration;
- a total evaluator relative to an oracle family;
- local maximum lemmas;
- a forest-stage operator; and
- a self-generated index with an explicit seed path.

Then seed inclusion, tower strictness, and diagonal comparison could be proved once and instantiated for System F, F_ω, F_κ, and future languages. This would reduce duplication and make future contenders easier to audit.

13.2 Prove quantitative embedding theorems

The current structural theorems show eventual domination of each individual W-definable tree. A stronger result would assign each enumerated tree a concrete level or path code and give an explicit threshold. This could support machine- checked lower bounds against named fast-growing hierarchies.

13.3 Study monotone sublanguages

Although the full reflected language is nonmonotone in its oracle, a syntactic positive-oracle fragment may admit a monotonicity theorem. Reflecting that fragment could enable cleaner ordinal iteration arguments while retaining a separate full-language maximum for raw size.

13.4 Calibrate the round operator

The main theoretical open problem is to identify an ordinal or reflection calculus faithfully represented by $\mathcal{R}$. Possible approaches include:

1. interpreting each W-tree stage as an iterated reflection theory;
2. extracting a proof-theoretic dilator from the syntax-to-tree operation;
3. proving upper and lower simulations between the Lean hierarchy and a standard collapsing-function system; and
4. determining the effect of self-feeding the numeric budget on the function hierarchy at a fixed ordinal index.

The literature on reflection ranks and reductions of model reflection to iterated syntactic reflection offers conceptual tools, but the BusyLean object languages and oracle-sensitive maxima require a bespoke theorem [9, 10, 11].

13.5 Build the stratified kind-universe successor

A major follow-up should formalize a predicative ladder $\Box_0 : \Box_1 : \Box_2 : \cdots$ above the impredicative term sort, with no self-typing universe. The research tasks are:

- syntax and de Bruijn operations for level-indexed kinds;
- a sound checker with fuel-bounded conversion;
- semantic domains and candidates indexed by levels;
- renaming and substitution across universe contexts;
- adequacy and strong normalization;
- a total evaluator and reflection operator; and
- comparison against Contender 11, preferably using `F11` as oracle.

13.6 Mechanize the metatheory of the audit itself

The closure command is a metaprogram that inspects the environment. One can go further by generating a signed manifest of declaration names, source hashes, Lean version, build timings, and axiom reports. This would make contest entries reproducible artifacts rather than informal snapshots.

14 Conclusion

Contender 11 is designed as the strongest low-risk successor available from the current BusyLean components. It does not attempt to win by arithmetic padding. Instead, it lifts the incumbent's kind-polymorphic reflection to a family-aware forest stage, traverses the self-generated index of all W-language-definable trees, iterates complete rounds with self-fed budgets, and diagonalizes through a number of rounds supplied by the predecessor construction itself.

The exact mathematical comparison is short because the architecture was chosen for explicit inclusions:

$$C_{10} = B(42) < F_{11}(42) < F_{11}(F_{11}(42)) = C_{11}.$$

No monotonicity of oracle reflection is assumed. Every component remains computable in principle, and the definition avoids all earlier contender constants. The remaining gate is empirical and formal rather than conceptual: run Lean 4.34.0, resolve any minor elaboration differences, verify the axiom and dependency audits, and record the build budgets. Until that is done, this is an integration-ready research proposal; after it is done, it is a natural Contender 11 for the repository.

A Core Lean definition

For convenient review, the complete proposed definition module is reproduced below.

```
import BusyLean.Contender10

namespace BusyLean

def reflect11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum
      (Fk.fvalue (packFamily fs) n)
      (Fk.ctmsUpTo n))

def stage11 (seed : W.Family) : BigNumber.WTree Nat → Nat → Nat
  | .leaf   => reflect11 seed
  | .node g => reflect11 (fun i => stage11 seed (g i))

def round11 (f : Nat → Nat) (n : Nat) : Nat :=
  stage11 (fun _ => f) index n

def family10 (n : Nat) : Nat := F10 (F10 n)

def tower11 : Nat → Nat → Nat
  | 0     => family10
  | k + 1 => fun n => round11 (tower11 k) (tower11 k n)

def F11 (n : Nat) : Nat :=
  tower11 (family10 n + 1) (family10 n)

def contender_11 : Nat := F11 (F11 42)

end BusyLean
```

B Theorem dependency graph

Declaration	Main dependencies and role
`lt_reflect11`	arithmetic form of the maximum
`oracle_lt_reflect11`	prefix membership + outer maximum
`fvalue_lt_reflect11`	list-maximum membership + outer maximum
`stage11_child_lt`	visible oracle entry at a parent node
`seed_lt_round11`	leaf $\to$ depth-zero join $\to$ global index
`tower11_step_lt`	round input bound at input $\mathcal{T}_k(n)$
`family10_lt_tower11_succ`	induction and transitivity over tower levels
`input_lt_family10`	two applications of Contender 10's input theorem
`family10_lt_F11`	base-family growth plus positive tower level
`input_lt_F11`	$n < B(n) < F_{11}(n)$
`contender_10_lt_contender_11`	family domination at 42 plus the outer diagonal

C Reproducibility commands

```
# In the BusyLean repository root, after copying the package files:
python3 scripts/check11.py

# Individual checks:
lake build BusyLean.Contender11
lake build BusyLean.Comparison11
lake env lean test/Contender11Audit.lean

# Article build from this package:
cd article
latexmk -pdf -interaction=nonstopmode -halt-on-error contender11.tex
```

D Acceptance checklist

1. Both Lean modules elaborate under exactly Lean 4.34.0.
2. `contender_11` is accepted as a computable `Nat` definition.
3. The required strict comparison is kernel-checked without `sorry`.
4. Every printed axiom set is a subset of {`propext`, `Quot.sound`}.
5. The transitive closure of `contender_11` contains no earlier contender constant.
6. Definition time is at most 15 seconds under the repository protocol.
7. Proof time is at most 60 seconds under the repository protocol.
8. The main README and article state measured timings, not estimates.
9. No one attempts to evaluate the contender.

References

[1] V. Reshetnikov and contributors. *BusyLean: name the biggest number, in Lean.* https://github.com/VladimirReshetnikov/BusyLean, inspected October 2026.

[2] BusyLean contributors. *Contender 6 and forest reflection modules.* Files `BusyLean/Reflection.lean`, `BusyLean/Contender6.lean`, and `BusyLean/Comparison.lean` in [1].

[3] BusyLean contributors. *Contender 9: System F_ω reflected along the W-language's own trees.* Files `BusyLean/Contender9.lean` and `BusyLean/Comparison9.lean` in [1].

[4] BusyLean contributors. *Contender 10: kind polymorphism.* File `BusyLean/Contender10.lean` in [1].

[5] BusyLean contributors. *Contender 10 comparison theorems.* File `BusyLean/Comparison10.lean` in [1].

[6] C. Roux and contributors. *name-the-biggest-number.* https://github.com/codyroux/name-the-biggest-number.

[7] S. Aaronson. Who can name the bigger number? https://www.scottaaronson.com/writings/bignumbers.html.

[8] H. Geuvers and M.-J. Nederhof. Modular proof of strong normalization for the Calculus of Constructions. *Journal of Functional Programming*, 1(2):155–189, 1991. doi:10.1017/S0956796800020037.

[9] F. Pakhomov and J. Walsh. Reflection ranks and ordinal analysis. *Journal of Symbolic Logic*, 86(4):1350–1384, 2021. doi:10.1017/jsl.2020.9; arXiv:1805.02095.

[10] F. Pakhomov and J. Walsh. Reducing ω-model reflection to iterated syntactic reflection. *Journal of Mathematical Logic*, 23(2), 2023. doi:10.1142/S0219061322500015; arXiv:2103.12147.

[11] M. Rathjen. Well-ordering principles in proof theory and reverse mathematics. arXiv:2010.12453, 2020.